UNIT-2 Functions, Exception, Modules and Files

• Function:

- A function is a block of reusable code that performs a specific task.
- It is defined using the `def` keyword and can be called independently.
- Functions can take arguments and return values.
- Example:

```
def greet(name):
    return f"Hello, {name}!"
print(greet("Alice"))
```

• Method:

- A method is a function that is associated with an object and is called on it. • It operates on data contained within the object and is defined inside a class. • Methods are invoked using the syntax: `object.method()`.
- Example:

```
class Person:
    def init (self, name):
        self.name = name

    def greet(self):
        return f"Hello, {self.name}!"

p = Person("Alice")
print(p.greet())
```

Difference Between a Function and a Method :

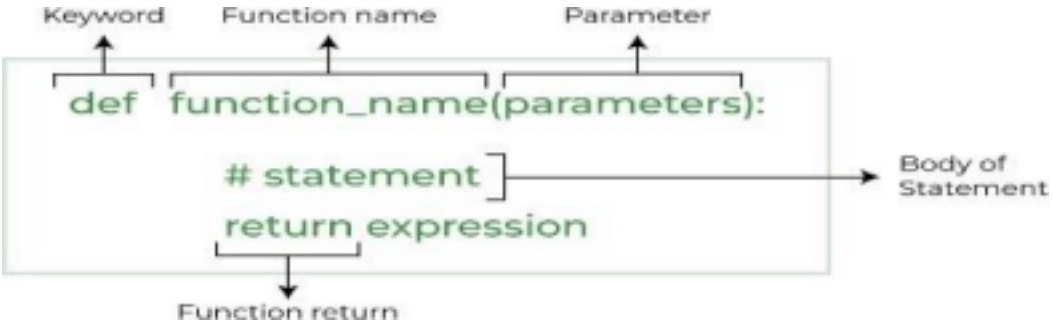
Aspect	Function	Method
Definition	A standalone block of reusable code defined using the `def` keyword.	A function associated with an object, defined inside a class.
Calling	Called independently using its name, e.g., `function_name()`.	Called on an object, e.g., `object.method_name()`.
Context	Not bound to any specific object or class.	Operates on the object it is called on and can access its data.
Accessto Data	Can only access data explicitly passed to it as arguments.	Can access the instance (`self`) and its attributes directly.
Definition	Defined outside of classes.	Defined within a class.

Location		
Example		

Aspect	Function	Method
Function Example		
	def add(a, b):	class Calculator:
	return a + b	def add(self, a, b):
	print(add(2, 3)) # Outputs: 5	return a + b
		calc = Calculator()
		print(calc.add(2, 3)) # Outputs: 5

Defining a Function :

- **Purpose**: A function is a reusable block of code designed to perform a specific task.



- **Syntax**:

```
def function_name(parameters):
    """Optional docstring"""
    # Function body
    return value
```

- **Components**:

1. `def` **Keyword**: Used to define the function.
2. **Function Name**: Should be descriptive and follow Python's naming conventions (e.g., snake_case).
3. **Parameters (Optional)**: Variables that accept input values.
4. **Docstring (Optional)**: Describes the purpose of the function.
5. **Body**: Contains the logic of the function.
6. `return` **Statement (Optional)**: Specifies the value to return to the caller.

• **Example**:

```
def greet(name):
    """Greets a person with their name."""
    return f"Hello, {name}!"
```



```
print(greet("Alice")) # Output: Hello, Alice!
```

Calling a Function

• A function is called by using its name followed by parentheses.
• If the function accepts arguments, passthe required values within the parentheses. •
Syntax:

```
function_name(arguments)
```

• **Example**:

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Alice")) # Output: Hello, Alice!
```

Returning Results from a Function

• The `return` statement is used to send the result of the function back to the caller. •
A function can:

1. Return a single value.
2. Return multiple values (as a tuple).
3. Return nothing (`None`).

• **Syntax**:

```
return value
```

• **Examples**:

1. Returning a Single Value:

```
def square(num):
    return num ** 2
```

```
result = square(4)
print(result) # Output: 16
```

2. Returning Multiple Values:

```
def get_coordinates():
    return 10, 20

x, y = get_coordinates()
print(x, y) # Output: 10 20
```

Returning Multiple Values from a Function:

- **Purpose**: A function can return multiple values, which is useful when more than one piece of information is required as output.

- **How It Works**:
  - Multiple values are returned as a tuple.
  - These values can be unpacked into individual variables or used directly.
- **Syntax**:

```
def function_name():
    return value1, value2, value3
```

- **Example**:

```
def calculate(x, y):
    sum_ = x + y
    diff = x - y
    prod = x * y
    return sum_, diff, prod

result = calculate(10, 5)
print(result) # Output: (15, 5, 50)

# Unpacking the values
sum_, diff, prod = calculate(10, 5)
print(sum_, diff, prod) # Output: 15 5 50
```

Functions as First-Class Objects:

- **Definition**:
  In Python, functions are treated as first-class objects, meaning they can be assigned to variables, passed as arguments, returned from other functions, and stored in data structures.
- **Characteristics**:
  1. **Assigned to Variables**:
     Functions can be assigned to variables, which can then be used to call

the function.

```
def greet(name):
    return f"Hello, {name}!"

say_hello = greet
print(say_hello("Alice")) # Output: Hello, Alice!
```

2. **Passed as Arguments**:
   Functions can be passed as arguments to other functions.

```
def square(x):
    return x ** 2

def apply_function(func, value):
    return func(value)

print(apply_function(square, 4)) # Output: 16
```


3. **Returned from Functions**:
   Functions can be returned by other functions.

```
def outer_function():
    def inner_function():
        return "Hello from inner function!"
    return inner_function

func = outer_function()
print(func()) # Output: Hello from inner function!
```

4. **Stored in Data Structures**:
   Functions can be elements in lists, dictionaries, or other structures.

```
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

operations = {"add": add, "subtract": subtract}
print(operations["add"](10, 5)) # Output: 15
```

Formal and Actual Arguments:

Formal Arguments

- **Definition**: The placeholders or variables defined in the function's signature that receives values when the function is called.
- **Purpose**: Define what inputs the function expects.

• **Example**:

```
def greet(name): # `name` is the formal argument
    return f"Hello, {name}!"
```

Actual Arguments

• **Definition**: The real values or variables passed to the function when it is called. • **Purpose**: Provide the specific data for the function to operate on.
• **Example**:

```
print(greet("Alice")) # "Alice" is the actual argument
```

Differences

Aspect	Formal Arguments	Actual Arguments
Definition	Variables in the function definition.	Values provided during a function call.

Aspect	Formal Arguments	Actual Arguments
Scope	Exist only inside the function.	Exist outside the function and are passed in.
Example	`def add(a, b):` (a, b are formal)	`add(5, 3)` (5, 3 are actual)

Types of Actual Arguments

1. **Positional Arguments**: Matched based on their position.

```
def add(a, b):
    return a + b

print(add(5, 3)) # Positional: 5 -> a, 3 -> b
```

2. **Keyword Arguments**: Matched based on parameter names.

```
print(add(b=3, a=5)) # Keyword: b=3, a=5
```

3. **Default Arguments**: Formal arguments with predefined default values.

```
def greet(name="Guest"):
    return f"Hello, {name}!"
```

```
print(greet()) # Output: Hello, Guest!
```

4. **Variable-Length Arguments**: Allow passing arbitrary numbers of arguments.

```
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3, 4)) # Output: 10
```

Positional Arguments:

- **Definition**:
  Positional arguments are passed to a function in the same order as the parameters are defined in the function's signature. The position of each argument determines which parameter it corresponds to.
- **Features**:
  1. Matched based on their order in the function call.
  2. All positional arguments must be provided unless default values are defined for the parameters.
  3. Positional arguments are the most common way to pass values to a function.
- **Syntax**:

```
def function_name(param1, param2):
```



```
    # Function body
function_name(arg1, arg2) # Positional arguments
```

- **Example**:

```
def add(a, b):
    return a + b

result = add(5, 3) # 5 is assigned to `a`, and 3 is assigned to `b`
print(result) # Output: 8
```

- **Advantages**:
  - Simple and intuitive for functions with a fixed number of parameters.

Caution:
  - Ensure that the order of arguments matches the intended parameters to avoid logic errors.

Keyword Arguments:

- **Definition**:
  Keyword arguments are passed to a function by explicitly specifying the parameter names along with their corresponding values in the function call.

• **Key Features**:

1. The order of arguments does not matter, as they are matched to parameters by name.
2. Keyword arguments improve code readability and avoid ambiguity when calling functions.
3. They can be used alongside positional arguments, but positional arguments must come first.

• **Syntax**:

```
def function_name(param1, param2):
    # Function body
function_name(param1=value1, param2=value2) # Keyword arguments •
```

Example:

```
def describe_person(name, age, city):
    return f"{name} is {age} years old and lives in {city}."

# Using keyword arguments
result = describe_person(age=25, city="New York", name="Alice")
print(result) # Output: Alice is 25 years old and lives in New York.
```

• **Notes**:

1. Keyword arguments are particularly useful when a function has many parameters or parameters with default values.
2. Mixing positional and keyword arguments is allowed, but positional arguments must appear first:


```
describe_person("Alice", city="New York", age=25) # Valid
```

• **Default Parameter Values**:

Functions often use keyword arguments with default values to make them optional:

```
def greet(name="Guest"):
    return f"Hello, {name}!"

print(greet()) # Output: Hello, Guest!
print(greet(name="Bob")) # Output: Hello, Bob!
```

• **Advantages**:

- Enhances function call clarity.
- Allows skipping some arguments when default values are defined.

Caution:

- Do not repeat the same parameter with both positional and keyword arguments in the same call:

```
describe_person("Alice", name="Bob") # Error: multiple values
for argument 'name'
```

Default Arguments:

- **Definition**:
  Default arguments are parameters that have predefined values specified in the function definition. If the caller does not provide a value for a default argument, the default value is used.
- **Key Features**:
  1. Default arguments make some parameters optional during a function call.
  2. Must be defined after all non-default (required) parameters.
  3. Default values are evaluated at the time the function is defined.
- **Syntax**:

```
def function_name(param1, param2=default_value):
    # Function body
```

- **Example**:

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Alice")) # Output: Hello, Alice!
print(greet("Bob", greeting="Hi")) # Output: Hi, Bob!
```

- **Advantages**:
  - Simplifies function calls.
  - Useful for handling cases where a parameter frequently has the same value.

Caution:

- Avoid mutable default arguments like lists or dictionaries, as they can lead to unexpected behavior:

```
def add_to_list(item, my_list=[]):
    my_list.append(item)
    return my_list

print(add_to_list(1)) # Output: [1]
print(add_to_list(2)) # Output: [1, 2] (unexpected!)
```

- Correct way:

```
def add_to_list(item, my_list=None):
    if my_list is None:
        my_list = []
    my_list.append(item)
    return my_list
```

Variable-Length Arguments :

- **Definition**:
  Variable-length arguments allow a function to accept an arbitrary number of arguments, making it flexible to handle different amounts of data.
- **Types**:
  1. **`*args` (Non-keyword Variable-Length Arguments)**:
     - Collects additional positional arguments into a tuple.
       - Used when the number of positional arguments is not fixed.
  2. **`**kwargs` (Keyword Variable-Length Arguments)**:
     - Collects additional keyword arguments into a dictionary.
       - Used when the number of keyword arguments is not fixed.
- **Syntax**:
  - **`*args`**:

    ```
    def function_name(*args):
        # Function body
    ```

  - **`**kwargs`**:

    ```
    def function_name(**kwargs):
        # Function body
    ```

Example 1: `*args` (Positional Arguments):

```
def add_numbers(*args):
    return sum(args)

print(add_numbers(1, 2, 3)) # Output: 6
print(add_numbers(5, 10, 15, 20)) # Output: 50
```


Example 2: `**kwargs` (Keyword Arguments):

```
def display_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

display_info(name="Alice", age=25)
# Output:
# name: Alice
# age: 25
```

Example 3: Mixing `*args` and `**kwargs`:

- When using both `*args` and `**kwargs`, `*args` must appear before `**kwargs` in the function signature.

```
def display_data(*args, **kwargs):
    print("Positional Arguments:", args)
    print("Keyword Arguments:", kwargs)

display_data(1, 2, 3, name="Alice", age=25)
# Output:
# Positional Arguments: (1, 2, 3)
# Keyword Arguments: {'name': 'Alice', 'age': 25}
```

Advantages:

- Provides flexibility when the number of inputs varies.
- Useful for functions that need to handle a dynamic set of arguments.

Caution:

- Avoid confusion when passing both regular, keyword, and variable-length arguments in complex functions.

Feature	*args	**kwargs
Meaning	Used to pass multiple positional arguments	Used to pass multiple keyword arguments
Argument Type	Positional (values without names)	Keyword (values with names)
Data Structure	Stores arguments in a tuple	Stores arguments in a dictionary
Syntax	Uses a single asterisk *args	Uses double asterisk **kwargs

Feature	*args	**kwargs
Passing Values	Values are passed in order	Values are passed using key = value
Access		Access using keys (e.g.,

Method	**Access using index or loop (e.g., args[0])**	**kwargs['name'])**
Use Case	**When number of inputs is unknown**	**When named inputs are unknown**
Example	**fun(1, 2, 3)**	**fun(name="Steffy", age=22)**
Order in Function	**Comes before **kwargs if both are used**	**Comes after *args**

Local and Global Variables:

Local Variables

- **Definition**:
  Local variables are defined within a function and can only be accessed inside that function. They exist only during the function's execution.
- **Features**:
  1. Created when the function is called.
  2. Destroyed when the function ends.
  3. Cannot be accessed outside the function.
- **Example**:

```
def greet():
    name = "Alice" # Local variable
    print(f"Hello, {name}!")

greet() # Output: Hello, Alice!
print(name) # Error: NameError: name is not defined
```


Global Variables

- **Definition**:
  Global variables are defined outside any function or class and are accessible throughout the entire script or program, including inside functions (if explicitly accessed).

- **Features**:
    1. Declared outside functions.
    2. Can be accessed and modified inside functions using the `global` keyword.
- **Example**:

```
age = 30 # Global variable

def print_age():
    print(f"Age: {age}")

print_age() # Output: Age: 30
```

- **Modifying Global Variables**:
    - To modify a global variable inside a function, use the `global` keyword.

```
count = 0 # Global variable

def increment():
    global count
    count += 1

increment()
print(count) # Output: 1
```

Difference between Local and Global Variable

Sr. No.	Local Variable	Global Variable
1.	A variable which is declared inside a function is called as **local variable.**	A variable that is declared outside the function is called **global variable.**
2.	Accessibility of variable is only within the function in which it is declared.	Accessibility of variable is throughout the program. All functions in the program can access the global variable.
3.	The local variable is created when the function(in which it is defined) starts executing and it is destroyed when the execution is complete.	The global remains throughout the entire program execution.
4.	It is preferred as local variables are more reliable. Because the values cannot be changed outside the function.	It is generally not preferred as any function can change the values of global variable.

Key Differences:

Aspect	Local Variable	Global Variable
Scope	Limited to the function where it is defined.	Accessible throughout the script or

Aspect	Local Variable	Global Variable

		module.
Lifetime	Exists only during the function's execution.	Existsfor the entire runtime of the program.
Access	Cannot be accessed outside the function.	Can be accessed and modified globally.
Modification	Cannot modify global variables directly (unless specified).	Can be modified from any function using `global`.

The global Keyword:

- **Definition**:
  The `global` keyword is used to declare that a variable is global inside a function, meaning that it refers to the variable defined outside the function (in the global scope) rather than creating a new local variable.
- **Purpose**:
  The `global` keyword allows you to modify a global variable inside a function.
- **Key Features**:
  1. **Global Scope Access**: Without the `global` keyword, a function cannot modify global variables; it will create a local variable with the same name.
  2. **Modifying Global Variables**: The `global` keyword allows you to modify a global variable from within a function.
- **Syntax**:

```
global variable_name
```

- **Example 1: Using the `global` Keyword**:

```
count = 0 # Global variable

def increment():
    global count # Declare count as global
    count += 1

increment()
print(count) # Output: 1
```

- **Example 2: Without the `global` Keyword**:

```
count = 0

def increment():
    count = 5 # Local variable
```

```
    print(count) # Output: 5

increment()
print(count) # Output: 0 (global variable unchanged)
```

Passing a Group of Elements to a Function:

In Python, you can pass a group of elements (such as lists, tuples, or dictionaries) to a function by using either **args**, **kwargs**, or directly passing the collection.

1. Passing a List or Tuple as an Argument

- You can pass a list or a tuple directly to a function, and within the function, you can access the elements by indexing or iterating over them.
- **Example with a List**:

```
def print_elements(elements):
    for element in elements:
        print(element)

my_list = [1, 2, 3, 4]
print_elements(my_list)
```

- **Output**:

```
1
2
3
4
```

2. Using `*args` to Pass a Variable Number of Positional Arguments

- `*args` allows you to pass a variable-length list of positional arguments to a function. It packs all the arguments into a tuple.
- **Example with `*args`**:

```
def sum_numbers(*args):
    return sum(args)

result = sum_numbers(1, 2, 3, 4)
print(result) # Output: 10
```

3. Using `**kwargs` to Pass a Variable Number of Keyword Arguments

- `**kwargs` allows you to pass a variable-length list of keyword arguments to a function. It packs all keyword arguments into a dictionary.
- **Example with `**kwargs`**:

```
def print_info(**kwargs):
    for key, value in kwargs.items():
```



```
        print(f"{key}: {value}")

print_info(name="Alice", age=25, city="New York")
```

- **Output**:

```
name: Alice
age: 25
city: New York
```

4. Passing a Dictionary as an Argument

- You can pass a dictionary to a function, where the function can access the values using the keys.
- **Example with a Dictionary**:

```
def print_dict(info):
    for key, value in info.items():
        print(f"{key}: {value}")

person_info = {"name": "Alice", "age": 25, "city": "New York"}
print_dict(person_info)
```

- **Output**:

```
name: Alice
age: 25
city: New York
```

Recursive Functions:

- **Definition**:
  A recursive function is a function that calls itself in order to solve a problem. The function typically solves a smaller instance of the problem with each recursive call, eventually reaching a base case that stops the recursion.
- **Features**:
  1. **Base Case**: The base case is a condition that ends the recursion, preventing infinite calls.
  2. **Recursive Case**: The recursive case involves the function calling itself with a modified argument, bringing it closer to the base case.
  3. Recursive functions are useful for problems that can be broken down into smaller subproblems of the same type (e.g., factorial, Fibonacci sequence).
- **Syntax**:

```
def recursive_function(parameters):
    if base_case_condition:
        return base_case_value
    else:
        return recursive_function(modified_parameters)
```


- **Example 1: Factorial Function** A simple example of recursion is calculating the factorial of a number, which is the product of all positive integers less than or equal to the number.

```
def factorial(n):
    if n == 0 or n == 1: # Base case
        return 1
    else:
        return n * factorial(n - 1) # Recursive case

print(factorial(5)) # Output: 120
```

- **Example 2: Fibonacci Sequence** The Fibonacci sequence is defined as `F(n) = F(n-1) + F(n-2)` with base cases `F(0) = 0` and `F(1) = 1`.

```
def fibonacci(n):
    if n == 0: # Base case
        return 0
    elif n == 1: # Base case
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2) # Recursive case

print(fibonacci(6)) # Output: 8
```

- **Advantages of Recursion**:
    1. **Simpler Code**: Recursion can simplify the implementation of algorithms like tree traversal or factorial calculation, where an iterative solution might be more complex.
    2. **Intuitive for Certain Problems**: Recursive solutions are often more intuitive and closer to the mathematical definition of the problem.
- **Disadvantages of Recursion**:
    1. **Memory Usage**: Recursive calls consume memory on the call stack, which can lead to a stack overflow if the recursion is too deep.
    2. **Performance**: Recursive functions can be slower and less efficient due to function call overhead. In some cases, iterative solutions may be preferred.

Anonymous Functions or Lambdas:

- **Definition**:

An anonymous function is a function that is defined without a name. In Python, these functions are created using the `lambda` keyword. They are often used for short, simple operations, typically where a function is passed as an argument to higher-order functions like `map()`, `filter()`, or `reduce()`.

- **Syntax**:

```
lambda arguments: expression
```

- **Example**:


```
# Regular function
def square(x):
    return x * x

# Lambda function
square_lambda = lambda x: x * x
print(square_lambda(5)) # Output: 25
```

Using Lambdas with filter () Function:

- **Definition**:
  The `filter()` function in Python is used to filter elements of an iterable based on a condition specified in a function. It returns a filter object that can be converted into a list or another iterable. When combined with a lambda function, `filter()` allows for more concise, inline filtering.
- **Syntax**:

```
filter(function, iterable)
```

  - **`function`**: A function that tests each element in the iterable. If the function returns `True`, the element is included in the result.
  - **`iterable`**: The iterable (e.g., list, tuple) whose elements are to be filtered.
- **Using `lambda` with `filter()`**:
  You can pass a `lambda` function as the first argument to `filter()` to apply the filtering condition directly.
- **Example 1: Filtering Even Numbers**:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers)) # Output: [2, 4, 6]
```

- **Example 2: Filtering Strings with Length Greater Than 3**:

```
words = ['apple', 'bat', 'cat', 'elephant']
long_words = filter(lambda word: len(word) > 3, words)
print(list(long_words)) # Output: ['apple', 'elephant']
```

Using Lambdas with `map()` Function :

- **Definition**:
  The `map()` function in Python applies a given function to each item of an iterable (e.g., list, tuple) and returns an iterator that produces the results. When used with a `lambda` function, `map()` allows you to apply a transformation to each element of the iterable concisely.
- **Syntax**:

```
map(function, iterable)
```


  - **`function`**: A function that will be applied to each element in the iterable.
  - **`iterable`**: The iterable whose elements will be processed.
- **Using `lambda` with `map()`**:
  You can pass a `lambda` function as the `function` argument to `map()`. This is particularly useful when you need to apply a short, simple operation to each element of the iterable. • **Example 1: Squaring Numbers**: Suppose you want to square each number in a list. You can use `map()` with a `lambda` to achieve this.

```
numbers = [1, 2, 3, 4]
squared_numbers = map(lambda x: x ** 2, numbers)
print(list(squared_numbers)) # Output: [1, 4, 9, 16]
```

- **Example 2: Converting Strings to Uppercase**: You can use `map()` with a `lambda` to convert a list of strings to uppercase.

```
words = ['apple', 'banana', 'cherry']
uppercase_words = map(lambda word: word.upper(), words)
  print(list(uppercase_words)) # Output: ['APPLE', 'BANANA', 'CHERRY']
```

- **Example 3: Adding Two Lists Element-Wise**: You can use `map()` with `lambda` to apply an operation to corresponding elements from multiple iterables (lists).

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
summed = map(lambda x, y: x + y, list1, list2)
print(list(summed)) # Output: [5, 7, 9]
```

Advantages:

- **Simplification**: Using `lambda` with `map()` simplifies code when applying simple operations to each element in a list or other iterable.
- **Functionality**: It allows you to apply complex transformations without needing to write a full function or loop.

Example Comparison:

- Using `map()` with `lambda`:

```
numbers = [1, 2, 3, 4]
doubled_numbers = map(lambda x: x * 2, numbers)
print(list(doubled_numbers)) # Output: [2, 4, 6, 8]
```

- Without `map()`, using a `for` loop:

```
doubled_numbers = []
for x in numbers:
    doubled_numbers.append(x * 2)
print(doubled_numbers) # Output: [2, 4, 6, 8]
```


The `map()` version is more concise, especially when combined with `lambda` for simple operations.

Using Lambdas with reduce() Function :

- **Definition**:
  The `reduce()` function from the `functools` module in Python is used to apply a binary function (a function that takes two arguments) cumulatively to the items of an iterable, from left to right, so as to reduce the iterable to a single result.
- **Syntax**:

```
from functools import reduce
reduce(function, iterable, [initializer])
```

  - **`function`**: A function that takes two arguments. This function will be applied to the items of the iterable cumulatively.
  - **`iterable`**: The iterable whose elements will be reduced.
  - **`initializer`** *(optional)*: An initial value to start the reduction. If provided, it is used as the first argument in the first call to the function. If not provided, the first element of the iterable is used as the initial value.
- **Using `lambda` with `reduce()`**:
  You can pass a `lambda` function to `reduce()` to perform a specific operation cumulatively across the elements of the iterable.
- **Example 1: Summing Numbers**: You can use `reduce()` with a `lambda` to compute the sum of all elements in a list.

```
from functools import reduce

numbers = [1, 2, 3, 4]
sum_result = reduce(lambda x, y: x + y, numbers)
```

```
print(sum_result) # Output: 10
```

- Here, `reduce()` adds each pair of numbers in the list cumulatively. First, `1 + 2 = 3`, then `3 + 3 = 6`, and finally `6 + 4 = 10`.

- **Example 2: Finding the Maximum Value**: You can use `reduce()` with a `lambda` to find the maximum value in a list.

```
numbers = [1, 5, 3, 9, 2]
max_value = reduce(lambda x, y: x if x > y else y, numbers)
print(max_value) # Output: 9
```

- In this case, `reduce()` compares each pair of values and keeps the larger of the two.

- **Example 3: Multiplying All Elements**: You can use `reduce()` with a `lambda` to multiply all the elements of a list.

```
numbers = [1, 2, 3, 4]
product_result = reduce(lambda x, y: x * y, numbers)
```



```
print(product_result) # Output: 24
```

- This performs the cumulative multiplication: `1 * 2 = 2`, then `2 * 3 = 6`, and finally `6 * 4 = 24`.

Advantages:

- **Conciseness**: Using `lambda` with `reduce()` is very concise for reducing an iterable in a single line.
- **Functional Programming**: `reduce()` encourages functional programming patterns by reducing a sequence of data to a single result.

Example Comparison:

- Using `reduce()` with `lambda`:

```
from functools import reduce
numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, numbers)
print(result) # Output: 10
```

- Without `reduce()`, using a `for` loop:

```
numbers = [1, 2, 3, 4]
result = 0
for num in numbers:
    result += num
print(result) # Output: 10
```

Function Decorators:

- **Definition**:
  A function decorator is a function that takes another function as an argument and extends or alters its behavior without modifying the function itself. Decorators are often used to add functionality to existing functions in a modular and reusable way.
- **Syntax**:

```
@decorator_name
def function_name():
    # function body
```

The `@decorator_name` is syntactic sugar that simplifies applying a decorator to a function. It's equivalent to writing `function_name = decorator_name(function_name)`.

How Decorators Work:

1. **Decorator Function**: A decorator is a function that takes another function as input, does something (such as modifying it), and returns a new function.
2. **Wrapper Function**: Inside the decorator, the original function is wrapped with another function (often called a wrapper) that calls the original function and adds additional behavior before or after it.
3. **Return the Wrapped Function**: The decorator returns the wrapper function, which is then used in place of the original function.

Example 1: Basic Decorator

Let's create a simple decorator that prints a message before calling the function:

```
def
 simple_decorator(func):
 def wrapper():
     print("Before function call")
     func()
     print("After function call")
 return wrapper

 @simple_decorat
or def
say_hello():
 print("Hello!")

say_hello()
```

- **Output**:

```
Before function call
Hello!
After function call
```

In this example, `simple_decorator` is the decorator, and `say_hello` is the function being decorated. The `wrapper` function inside the decorator adds behavior before and after the original function call.

Example 2: Decorators with Arguments

Decorators can also be used with functions that accept arguments. Here's an example that uses a decorator to log the arguments passed to the function:

```
def log_arguments(func):
    def wrapper(*args, **kwargs):
        print(f"Arguments: {args}, {kwargs}")
        return func(*args, **kwargs)
    return wrapper

@log_arguments
def greet(name, age):
    print(f"Hello {name}, you are {age} years old.")
```



```
greet("Alice", 30)
```

- **Output**:

```
Arguments: ('Alice', 30), {}
Hello Alice, you are 30 years old.
```

Using functools.wraps() to Preserve Metadata:

When you decorate a function, the original function's metadata (such as its name and docstring) can be lost. To preserve it, you can use `functools.wraps()`.

```
from functools import wraps

def simple_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func. name }")
        return func(*args, **kwargs)
    return wrapper

@simple_decorator
def say_hello():
    """This is a greeting function."""
    print("Hello!")

print(say_hello. name ) # Output: say_hello
print(say_hello. doc ) # Output: This is a greeting function.
```

Generators :

- **Definition**:
  A generator in Python is a special type of iterable, like a list or a tuple, but it generates items lazily, one at a time, as they are needed. Instead of storing all the values in memory, a generator computes each value on the fly and yields it when required. This can be highly memory-efficient, especially when dealing with large datasets. • **Characteristics of Generators**:
    1. **Lazy Evaluation**: Generators generate values on the fly, meaning they don't store all the values in memory at once.
    2. **State Preservation**: Generators maintain their state between iterations, which allows them to resume execution from where they left off.
    3. **Use of `yield`**: The `yield` keyword is used in a generator function to return values. Unlike `return`, which exits a function, `yield` temporarily suspends the function's execution and sends a value back to the caller.

Creating a Generator

- **Using Generator Functions**:
  A generator function is defined like a normal function, but instead of `return`, it uses `yield` to yield values.

```
def count_up_to(max):
    count = 1
    while count <= max:
        yield count
        count += 1
```

- Here, `count_up_to` is a generator function that yields numbers from 1 to the given `max` value.

- **Using the Generator**: You can use the `for` loop or `next()` to retrieve values from a generator.

```
counter = count_up_to(5)
for num in counter:
    print(num)
```

- **Output**:

```
1
2
3
4
5
```

- **Using `next()`**: You can manually get the next value using the `next()` function:

```
counter = count_up_to(3)
print(next(counter)) # Output: 1
print(next(counter)) # Output: 2
print(next(counter)) # Output: 3
```

If you call `next()` again after the generator is exhausted, it will raise a `StopIteration` exception.

```
print(next(counter)) # Raises StopIteration
```

Advantages of Generators:

1. **Memory Efficiency**: Generators are more memory-efficient than lists because they generate items one at a time, without the need to store the entire collection in memory. 2. **Lazy Evaluation**: Values are produced only when requested, which can save time and processing when dealing with large datasets or infinite sequences.
3. **Cleaner Code**: Generators can replace complex iterators with simple, readable code.

Example 1: Fibonacci Sequence

A common example of using a generator is to create an infinite Fibonacci sequence:

```
def fibonacci():
    a, b = 0, 1
    while True:
```



```
        yield a
        a, b = b, a + b
```

You can retrieve Fibonacci numbers as needed:

```
fib = fibonacci()
for _ in range(5):
    print(next(fib)) # Output: 0 1 1 2 3
```

Example 2: Generator Expressions

Generator expressions are a concise way to create generators. They are similar to list comprehensions but use parentheses `()` instead of square brackets `[]`.

```
squares = (x ** 2 for x in range(5))
for square in squares:
    print(square)
```

- **Output**:

```
0
1
4
9
16
```

Comparison to Lists:

- **Lists**: Store all items in memory, so large lists can consume a lot of memory. • **Generators**: Compute each item on demand, making them more memory-efficient, especially for large or infinite sequences.

When to Use Generators:

- When dealing with large datasets that don't need to be stored all at once. • When creating infinite sequences where you only need to compute one value at a time (e.g., Fibonacci sequence).
- When you want to optimize memory usage or make your code more efficient.

Structured Programming:

- **Definition**:

Structured programming is a programming paradigm that emphasizes dividing a program into smaller, manageable parts using well-defined control structures. It aims to improve the clarity, quality, and development time of a program by making it easier to follow and understand. Structured programming involves the use of sequences, decisions (if-else), and loops, without relying on "go-to" statements that lead to unstructured jumps in the program flow.

Principles of Structured Programming:

1. **Modularization**:
   Breaking down a program into functions or procedures that each handle a specific task. This improves code reusability and readability.
2. **Top-Down Design**:
   Designing programs from the top down, starting with the high-level functionality and then breaking it into smaller tasks. Each task is implemented as a function or module. 3. **Control Structures**:
   Using clear, logical control structures like `if`, `else`, `while`, and `for` loops, and avoiding `goto` or similar constructs. These control structures ensure that the flow of execution is predictable and follows a logical sequence.
4. **Data Flow**:
   Data should be passed between functions via parameters, and results should be returned through return values, not global variables.
5. **Avoidance of Global Variables**:
   One of the goals of structured programming is to limit the use of global variables. Global variables can lead to unpredictable behavior and make it difficult to understand how data is being modified throughout the program.

Features of Structured Programming:

1. **Functions and Procedures**:
   Python encourages the use of functions for modularization. Functions break a program into smaller, manageable parts that can be reused and tested independently.

   ```
   def calculate_area(radius):
       return 3.14 * radius * radius
   ```

   This `calculate_area` function takes a `radius` as input and returns the area of a circle. By encapsulating this logic inside a function, you can use it anywhere in the program.

2. **Control Flow Statements**:
   - **Conditional Statements (`if-else`)**:
     Python uses `if`, `elif`, and `else` to make decisions based on conditions.

     ```
     def check_number(num):
         if num > 0:
     ```

```
        return "Positive"
    elif num == 0:
        return "Zero"
    else:
        return "Negative"
```

- **Loops (`for`, `while`)**:
  Loops allow repetitive execution of code until a condition is met.


```
def sum_of_numbers(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
```

3. **Avoiding `goto` Statements**:
   Structured programming discourages the use of `goto` or similar unstructured jumps (such as `break` and `continue` used excessively). Python avoids this by promoting clear control flow using loops and conditionals.
4. **Data Encapsulation**:
   Python allows for the encapsulation of data inside functions, which ensures that data is only modified in specific ways and improves readability and maintainability. Python also allows the use of classes to encapsulate both data and behavior, although object-oriented programming (OOP) extends structured programming.

Benefits of Structured Programming:

1. **Readability**:
   Programs are easier to read and understand because of clear and logical structure. 2. **Maintainability**:
   The modular approach makes it easier to maintain and update programs. Changes can be made to individual functions without affecting the entire program.
3. **Reusability**:
   Functions and modules can be reused in other programs or projects.
4. **Debugging and Testing**:
   Smaller modules are easier to test and debug. Errors are often localized within specific functions or blocks.
5. **Avoiding Spaghetti Code**:
   By organizing the program into functions and avoiding unstructured jumps (like `goto`), structured programming helps avoid "spaghetti code," where the flow of execution is hard to follow.

Example:

```
def main():
```

```
    num = int(input("Enter a number: "))
    result = check_number(num)
    print(f"The number is {result}")

def check_number(num):
    if num > 0:
        return "Positive"
    elif num == 0:
        return "Zero"
    else:
        return "Negative"

main()
```


Creating Our Own Modules:

- **Definition**:
  A module in Python is simply a file containing Python definitions and statements. It can contain functions, classes, variables, or runnable code. Modules allow you to logically organize your Python code and make it reusable in multiple programs.

Steps to Create a Module:

1. **Create a Python File**:
   A module is just a Python file with the `.py` extension. For example, you could create a file called `mymodule.py`.
2. **Define Functions, Classes, or Variables**:
   Inside the Python file, you can define functions, classes, and variables that you want to include in your module. Here's an example:

```
# mymodule.py

def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

3. **Use Your Module in Other Python Files**:
   Once your module is created, you can use it in other Python files by importing it using the `import` statement.

```
# main.py
import mymodule

result = mymodule.add(5, 10)
print(result) # Output: 15

greeting = mymodule.greet("Alice")
```

```
print(greeting) # Output: Hello, Alice!
```

In this example, the `mymodule` is imported, and its functions are accessed using the `mymodule.function_name` syntax.

Importing Specific Functions from a Module:

Instead of importing the entire module, you can import specific functions or classes using the `from` keyword:

```
# main.py
from mymodule import greet, add

print(add(5, 10)) # Output: 15
```



```
print(greet("Bob")) # Output: Hello, Bob!
```

This way, you don't have to prefix the function names with the module

name. **Using Aliases for Modules:**

You can also create an alias for your module, which is useful when the module name is long or if you want to make your code more concise:

```
# main.py
import mymodule as mm

print(mm.add(3, 7)) # Output: 10
```

Module Search Path:

When you import a module, Python looks for the module in several places, including:

1. The directory where the current script is located.
2. The directories listed in the `sys.path` variable, which includes directories like the standard library and site-packages.

If your module is located in a different directory, you can either add the path to `sys.path` or place the module in the same directory as your script.

Example of a Complete Module and Usage:

- **mymodule.py**:

```
# mymodule.py

def greet(name):
```

```
    return f"Hello, {name}!"

def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

- **main.py**:

```
# main.py
import mymodule

print(mymodule.greet("Alice")) # Output: Hello, Alice!
print(mymodule.add(5, 10)) # Output: 15
print(mymodule.multiply(4, 6)) # Output: 24
```


Special Module name_:

When you run a module directly, the special built-in variable `name` is set to `" main "`. This can be used to distinguish between when the module is being run directly or being imported:

```
# mymodule.py

def greet(name):
    return f"Hello, {name}!"

if name_== " main ":
    print("This is the main program!")
    greet("Alice")
```

If `mymodule.py` is run directly, the `if name_== " main ":` block will execute, but if it is imported into another module, it won't run that part of the code.

The Special Variable name_:

In Python, `name` is a special built-in variable that holds the name of the current module or script. It plays an important role in determining how a module is being used—whether it is being run directly or imported into another module.

Understanding `name`

1. **When a Script is Run Directly**:
   If a Python script is executed directly (not imported as a module), the value of `name` is set to `" main "`. This allows the script to perform certain actions only when it is being

run directly, rather than when it is being imported into another program. 2. **When a Script is Imported**:

If a Python script is imported as a module into another script, the value of `name` is set to the name of the module (i.e., the file name without the `.py` extension). This is useful for determining whether the script is being used as a module or being executed as the main program.

Example:

Example 1: Running a Script Directly

Consider the following Python file `example.py`:

```
# example.py

def say_hello():
    print("Hello from example.py!")

if name_== " main ":
```



```
    print("This is the main program!")
    say_hello()
```

- If you run `example.py` directly using the command `python example.py`, the output will be:

```
This is the main program!
Hello from example.py!
```

  - The `if name_== "_main ":` block is executed because `example.py` is being run directly. The value of `name` is `" main_"`, so the block under this condition is executed.

Example 2: Importing a Module

Now, suppose we import `example.py` into another script `main.py`:

```
# main.py
import example
```

- When you run `main.py` with the command `python main.py`, the output will be:

```
(No output)
```

  - Here, `example.py` is imported as a module. The `if name == " main ":` block in `example.py` is **not** executed, because `name` is set to `"example"` (the name of the module), not `" main "`.

Exceptions:

Errors in a Python Program:

In Python, errors can occur at different stages of program execution. Understanding the types of errors and how to handle them is crucial for writing effective Python programs. There are three main types of errors:

1. **Compile-Time Errors (Syntax Errors)**:
   - **Definition**: Compile-time errors, also known as syntax errors, occur when the Python interpreter encounters code that violates the syntax rules of the language. These errors prevent the program from being compiled or executed.
   - **Common Causes**: Misspelled keywords, incorrect indentation, unmatched parentheses or quotes, and other syntactic mistakes.
   - **Example**:

```
# SyntaxError example
if x = 10:
    print("x is 10")
```


     - **Error Message**: `SyntaxError: invalid syntax`
     - **Explanation**: The `=` operator should be `==` for comparison, but since `=` is an assignment operator, it causes a syntax error.
   - **Solution**: Correct the code to match proper Python syntax.

```
if x == 10:
    print("x is 10")
```

2. **Runtime Errors (Exceptions)**:
   - **Definition**: Runtime errors occur while the program is running and typically happen when the program tries to execute an operation that is not allowed or possible. These errors are also called exceptions, and they interrupt the normal flow of the program.
   - **Common Causes**: Division by zero, accessing an undefined variable, trying to open a non-existent file, and invalid operations like accessing elements in a list that don't exist.
   - **Example**:

```
# ZeroDivisionError example
a = 10
b = 0
result = a / b
```

     - **Error Message**: `ZeroDivisionError: division by zero`

- **Explanation**: The program attempts to divide by zero, which is mathematically undefined.

- **Solution**: Add error handling using `try` and `except` to catch the exception and handle it appropriately.

```
try:
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

- **Common Types of Runtime Errors**:
  - `IndexError`: Accessing an index that is out of range in a list.
  - `FileNotFoundError`: Trying to open a file that doesn't exist.
  - `TypeError`: Performing an operation on an object of the wrong type (e.g., adding a string to an integer).
  - `ValueError`: Passing an invalid value to a function.

3. **Logical Errors**:
   - **Definition**: Logical errors are mistakes in the program's logic that cause it to behave unexpectedly. Unlike syntax and runtime errors, logical errors do not stop the program from running, but they lead to incorrect results or unexpected behavior.
   - **Common Causes**: Incorrect algorithm or logic in the code, improper use of variables, wrong conditionals, or incorrect assumptions about data.
   - **Example**:


```
# LogicalError example
def calculate_area(radius):
    return radius + radius * 3.14 # Incorrect formula for area
of a circle

result = calculate_area(5)
print(result) # Output will be wrong
```

  - **Explanation**: The formula for the area of a circle should be `π * radius^2`, but the program uses an incorrect formula. This causes the program to produce the wrong result.
  - **Output**: `20.7` (incorrect result).
- **Solution**: Correct the algorithm or logic.

```
def calculate_area(radius):
    return 3.14 * radius * radius # Correct formula for area of  a
circle
```

- **Identifying and Fixing Logical Errors**:
  - Logical errors are often the hardest to detect because they don't crash the program. They may only show up during specific conditions or with particular inputs.

- Debugging tools, such as `print()` statements or using an interactive debugger, can help identify where the logic goes wrong.

Exceptions, Exception Handling, and Types of Exceptions:

In Python, **exceptions** are special objects that represent errors that can occur during the execution of a program. Exceptions interrupt the normal flow of execution and must be handled in order to ensure that the program doesn't crash.

1. What Are Exceptions?

An exception is an event that occurs during the execution of a program, disrupting its normal flow. It is typically caused by errors like dividing by zero, accessing an invalid index in a list, or trying to open a file that doesn't exist. When an exception occurs, Python creates an exception object and halts further execution unless the exception is caught and handled.

2. Exception Handling :

Python provides a mechanism for handling exceptions, which prevents the program from terminating unexpectedly. This is done using the `try`, `except`, `else`, and `finally` blocks.

- **`try` block**: The code that may cause an exception is placed inside the `try` block.
- **`except` block**: The `except` block catches and handles the exception if it occurs.
- **`else` block**: The `else` block runs if no exceptions are raised in the `try` block.

- **`finally` block**: The `finally` block is always executed, regardless of whether an exception occurred or not. It is typically used for clean-up operations (e.g., closing files).

Syntax of Exception Handling:

```
try:
    # Code that may raise an exception
    risky_code()
except SomeException as e:
    # Code to handle the exception
    print(f"An error occurred: {e}")
else:
    # Code to run if no exceptions occurred
    print("No errors occurred.")
finally:
    # Clean-up code (runs no matter what)
    print("This will always execute.")
```

3. Example of Exception Handling

```
try:
    x = 10 / 0 # Division by zero
except ZeroDivisionError as e:
    print(f"Error: {e}") # Catches ZeroDivisionError
else:
    print("This won't run because of the exception.")
finally:
    print("This block runs no matter what.")
```

- **Output**:

```
Error: division by zero
This block runs no matter what.
```

4. Types of Exceptions

Python has many built-in exceptions. Some of the most common ones are:

1. **`ZeroDivisionError`**:
   - Occurs when a division or modulo operation is performed with zero as the divisor.
   - Example:

```
10 / 0 # Raises ZeroDivisionError
```

2. **`IndexError`**:
   - Raised when trying to access an index that is out of the range of a list or tuple.
   - Example:

```
lst = [1, 2, 3]
print(lst[5]) # Raises IndexError
```


3. **`KeyError`**:
   - Raised when a dictionary key is not found.
   - Example:

```
my_dict = {'a': 1, 'b': 2}
print(my_dict['c']) # Raises KeyError
```

4. **`FileNotFoundError`**:
   - Raised when trying to open a file that does not exist.
   - Example:

```
open('non_existent_file.txt', 'r') # Raises FileNotFoundError
```

5. **`TypeError`**:

- Raised when an operation or function is applied to an object of an inappropriate type.
- Example:

```
'string' + 10 # Raises TypeError (can't add string and integer)
```

6. **`ValueError`**:
   - Raised when a function receives an argument of the correct type but an inappropriate value.
   - Example:

```
int('abc') # Raises ValueError
```

7. **`AttributeError`**:
   - Raised when an invalid attribute reference or assignment is made to an object.
   - Example:

```
x = 10
x.append(5) # Raises AttributeError (int object has no attribute
'append')
```

8. **`ImportError`**:
   - Raised when an import statement fails to find the module.
   - Example:

```
import non_existent_module # Raises ImportError
```

9. **`AssertionError`**:
   - Raised when an `assert` statement fails.
   - Example:

```
assert 1 == 2 # Raises AssertionError
```


5. Handling Multiple Exceptions

You can handle multiple exceptions in a single `try` block using multiple `except` clauses.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError as e:
    print("Cannot divide by zero.")
except ValueError as e:
    print("Invalid input! Please enter a number.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

```
else:
    print(f"Result is {result}")
finally:
    print("Execution completed.")
```

6. Raising Exceptions

You can also raise exceptions manually using the `raise` statement, which is useful for enforcing conditions in your program.

```
def check_age(age):
    if age < 18:
        raise ValueError("Age must be 18 or older.")
    return f"Age is {age}."

try:
    print(check_age(16))
except ValueError as e:
    print(f"Error: {e}")
```

- **Output**:

```
Error: Age must be 18 or older
```

The except Block in :

In Python, the `except` block is used to catch and handle exceptions (errors) that occur in the `try` block. It allows you to define how the program should respond when specific exceptions are encountered. If an exception occurs in the `try` block, Python immediately jumps to the corresponding `except` block and executes the code within it.

Syntax of the `except` Block

```
try:
    # Code that may raise an exception
    risky_code()
```



```
except SomeException as e:
    # Code to handle the exception
    print(f"An error occurred: {e}")
```

Example of Using the `except` Block

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

```
except ValueError as e:
    print(f"Invalid input! {e}")
else:
    print(f"Result: {result}")
finally:
    print("Execution completed.")
```

- If you enter `0`, the `ZeroDivisionError` is caught.
- If you enter a non-numeric value, the `ValueError` is caught.
- If the input is valid, the `else` block will run, displaying the result.
- The `finally` block always runs at the end.

The assert Statement:

The `assert` statement is used in Python to test if a condition is true. If the condition is **true**, the program continues to execute as usual. If the condition is **false**, an `AssertionError` is raised, which can be caught in an `except` block or left to terminate the program. This can be helpful for debugging and ensuring that certain conditions hold during the execution of a program.

Syntax of `assert`

```
assert condition, "Error message"
```

- **condition**: This is the expression you want to test. If it evaluates to `True`, the program continues. If it evaluates to `False`, an `AssertionError` is raised.
- **"Error message"** (optional): A string that is provided when the assertion fails. This message can be useful for debugging purposes to explain why the assertion failed.

How `assert` Works

- When the condition is `True`, the `assert` statement does nothing and the program continues as expected.
- When the condition is `False`, an `AssertionError` is raised, and the optional error message is displayed.

Example of `assert` in Action

```
x = 5
assert x > 0, "x should be greater than 0" # Passes, since 5 > 0

y = -3
assert y > 0, "y should be greater than 0" # Raises AssertionError with the
message: "y should be greater than 0"
```

User-Defined Exceptions:

In Python, you can create your own exceptions by defining a custom exception class. This allows you to raise exceptions specific to your application's needs. Custom exceptions help make your code more readable and maintainable, especially when you want to signal specific error conditions that don't fall under the standard exceptions.

Defining a User-Defined Exception

To define a user-defined exception, you need to create a new class that inherits from the built-in `Exception` class or any of its subclasses. The new class can include additional functionality, such as custom error messages or logging.

Syntax:

```
class CustomError(Exception):
    def init (self, message):
        super(). init (message)
        self.message = message
```

- **`CustomError`**: This is the name of the custom exception class.
- **`init` method**: This is the constructor, which allows you to pass custom messages or additional information when the exception is raised.
- **`super(). init (message)`**: This calls the base `Exception` class constructor to initialize the exception with the provided message.

Raising a User-Defined Exception

Once a custom exception is defined, you can raise it using the `raise` keyword.

Example of Raising a Custom Exception:

```
class NegativeValueError(Exception):
    def init (self, message):
        super(). init (message)

def calculate_area(radius):
    if radius < 0:
        raise NegativeValueError("Radius cannot be negative")
    return 3.14 * radius * radius

try:
```



```
    area = calculate_area(-5)
except NegativeValueError as e:
    print(f"Error: {e}")
```

In this example:

- The custom exception `NegativeValueError` is defined.
- The `calculate_area` function raises this exception if the radius is negative. • The exception is caught and handled in the `except` block.

Logging Exceptions:

Logging exceptions in Python is a powerful way to keep track of errors that occur in your application. Instead of simply printing error messages or using `print()` for debugging, logging provides a structured way to capture, store, and analyze errors over time. Python's `logging` module allows you to log detailed information about exceptions, which is crucial for debugging and monitoring.

Why Log Exceptions?

- **Traceability**: Log files provide a record of exceptions that occurred, along with detailed context (such as timestamps, error messages, and stack traces).
- **Error Tracking**: It helps in tracking recurring errors and understanding how often they occur. • **Remote Monitoring**: Logs can be sent to centralized logging systems, where they can be monitored and analyzed in real-time.
- **Debugging**: Detailed logs help developerstroubleshoot issues in production environments.

Setting up Logging

To start logging exceptions, you need to configure the `logging` module. The basic setup involves specifying the log level, the format of the logs, and where the logs should be stored.

Basic Setup Example:

```
import logging

# Configuring logging
logging.basicConfig(
    level=logging.ERROR, # Minimum log level to capture (ERROR, WARNING,
INFO, DEBUG)
    format='%(asctime)s - %(levelname)s - %(message)s', # Log format
    filename='app.log', # Log file to store the logs
    filemode='a' # Append to the log file
)

# Example of logging an exception
try:
    x = 10 / 0 # Will raise a ZeroDivisionError
except ZeroDivisionError as e:
```



```
    logging.error("An exception occurred: %s", e, exc_info=True) # Log the
exception with traceback
```

Features of Logging Exceptions:

1. **`exc_info=True`**:
   - This flag in the `logging.error()` method is crucial. When set to `True`, it includes the stack trace in the log, which helps identify where the exception occurred.

   Example:

   ```
   try:
       x = int("abc")
   except ValueError as e:
        logging.error("Invalid input encountered: %s", e, exc_info=True)
   ```

2. **Log Levels**: The `logging` module supports several log levels, each indicating the severity of the message:
   - **DEBUG**: Detailed information for diagnostic purposes.
   - **INFO**: Informational messages that confirm the program is working as expected.
   - **WARNING**: Indications of potential issues or unexpected behavior.
   - **ERROR**: Errors that occurred during the execution, but the program can continue.
   - **CRITICAL**: Severe errorsthat cause the program to stop.

   Example of logging with different levels:

   ```
   logging.debug("This is a debug message")
   logging.info("This is an info message")
   logging.warning("This is a warning message")
   logging.error("This is an error message")
   logging.critical("This is a critical message")
   ```

Example of Logging an Exception with Traceback

```
import logging

# Configuring logging
logging.basicConfig(
    level=logging.ERROR,
    format='%(asctime)s - %(levelname)s - %(message)s',
    filename='app.log',
    filemode='a'
)

# Function that raises an exception
def risky_function():
    try:
        x = 10 / 0
    except ZeroDivisionError as e:
         logging.error("A division by zero occurred: %s", e, exc_info=True)
```

```
# Call the function that logs the exception
risky_function()
```

Files:

Working with Files:

In Python, handling files is done using built-in functions and methods. There are different types of files you can work with, and Python provides functionality to open, read, write, and close files in a variety of formats.

Types of Files in Python

1. **Text Files**: These files contain readable characters and store data in a human-readable format (ASCII, UTF-8, etc.). Python allows reading and writing to text files using standard input/output methods.
2. **Binary Files**: These files store data in binary format (non-readable by humans) such as images, audio files, or executable files. Binary files are handled differently from text files and require opening the file in binary mode (`'rb'`, `'wb'`).

Opening a File

To open a file, Python provides the `open()` function. You can specify the file path and the mode in which the file should be opened. Common file modes include:

- `'r'`: Read (default mode, opens the file for reading).
- `'w'`: Write (opens the file for writing, creates the file if it does not exist).
- `'a'`: Append (opens the file for appending, creates the file if it does not exist). •
  `'rb'`: Read in binary mode.
- `'wb'`: Write in binary mode.

Example:

```
# Opening a file in read mode
file = open('example.txt', 'r')
```

Closing a File

It is important to close the file once operations are done to free up system resources. Use the `close()` method for this purpose.

Example:

```
file.close()
```

Working with Text Files Containing Strings

You can read and write strings to text files using the `read()`, `write()`, or `writelines()` methods. Similarly, you can read all the contents of a file or line by line.

Example:

```
# Writing text to a file
with open('example.txt', 'w') as file:
    file.write("Hello, world!\nThis is a text file.")

# Reading text from a file
with open('example.txt', 'r') as file:
    content = file.read()
    print(content)
```

Knowing Whether a File Exists or Not

Before working with a file, it is often useful to check whether the file exists. You can use the `os.path.exists()` method from the `os` module to check if a file exists.

Example:

```
import os

if os.path.exists('example.txt'):
    print("File exists")
else:
    print("File does not exist")
```

Working with Binary Files

For binary files, the reading and writing operations differ from text files. You use `'rb'` and `'wb'` modes for reading and writing binary data.

Example:

```
# Writing binary data to a file
data = b"Hello, world!" # byte string
with open('example.bin', 'wb') as file:
    file.write(data)

# Reading binary data from a file
with open('example.bin', 'rb') as file:
    binary_data = file.read()
    print(binary_data)
```

The `with` Statement

The `with` statement in Python is used for managing resources like files, ensuring that they are properly opened and closed. It automatically closes the file once the block of code inside the `with` statement is executed, eliminating the need to manually call `close()`.

Example:

```
with open('example.txt', 'r') as file:
    content = file.read()
    print(content)
# No need to call file.close(), it's done automatically
```

Pickle :

The `pickle` module is used for serializing and deserializing Python objects, which allows you to save complex data structures (like lists, dictionaries, etc.) to a file and later reload them.

Example:

```
import pickle

# Serialize data and save it to a file
data = {'name': 'Alice', 'age': 25}
with open('data.pkl', 'wb') as file:
    pickle.dump(data, file)

# Deserialize data from the file
with open('data.pkl', 'rb') as file:
    loaded_data = pickle.load(file)
    print(loaded_data)
```

The `seek()` and `tell()` Methods

1. **`seek(offset, whence)`**: This method moves the file pointer to a specified position. The `offset` is the number of bytes to move, and `whence` determines from where the position is moved:
    - `0` (default): Moves from the beginning of the file.
    - `1`: Moves from the current position.
    - `2`: Moves from the end of the file.
2. **`tell()`**: This method returns the current position of the file pointer.

Example:

```
with open('example.txt', 'r') as file:
    file.seek(5) # Move to the 5th byte
    print(file.tell()) # Get the current position (5)
    content = file.read()
    print(content)
```